

Farm Management System

Complete Technical Documentation

Project	Farm Intelligent Management Desktop App
Framework	JavaFX 21 (Maven build)
Pattern	MVC + Service Layer
School	Ecole Nationale Supérieure d'Informatique (ESI Alger)
Level	2CP -- Academic Year 2025/2026
Generated	19 May 2026

Table of Contents

Table of Contents	2
1. Project Overview	4
2. Architecture and MVC Pattern	5
3. Package: Additional_classes	6
4. Package: Alerts	7
5. Package: Animals	10
6. Package: Entities	14
7. Package: Farm	18
8. Package: Reports	20
9. Package: Sensors	23
10. Package: ZONES	27
11. Layer: Services	29
12. Layer: Controllers (JavaFX)	34
13. Layer: Views (FXML)	39
14. Layer: Styles (CSS)	43
15. Utilities and Entry Point	45
16. Class Relationship Summary	47
17. Data Flow Examples	48
Scenario A — Farm Manager Adds a New Crop Zone	48
Scenario B — A Soil Sensor Reading Exceeds its Threshold	48
Scenario C — Farm Manager Generates a Production Report	48

1. Project Overview

The Farm Management System is a desktop application built with JavaFX 21 and Maven. It provides an integrated platform for managing all aspects of a modern farm: livestock zone management and animal health monitoring, crop field tracking from sowing to harvest, aquaculture species population and harvest records, a distributed sensor network (bio, environmental, soil, water, and GPS collar sensors), a real-time alert system with severity classification, structured production reports, and a time-step simulation engine that advances farm state day by day.

Layer	Technology	Role
Model	Java domain classes	Represent farm entities, sensors, alerts, reports
Service	Java singleton classes	Business logic, data filtering, auto-save
Controller	JavaFX + FXML	Handle UI events, call services, update views
View	FXML + CSS	Declare UI layout, bind to controller methods
Utility	Java (SceneManager etc)	SPA navigation, persistence, random data generation

2. Architecture and MVC Pattern

The application follows a strict MVC + Service Layer pattern. The Model layer comprises all domain Java classes (Animals, ZONES, Sensors, Reports, etc.) that hold data and enforce invariants through constructor validation and immutable return types. The Service layer (com.example.services) provides singleton facades over the Model, encapsulating business logic so that controllers never manipulate domain objects directly. Each Controller (com.example.controllers) is paired with exactly one FXML view file; it reads from Services, updates JavaFX controls, and handles user events via @FXML-annotated methods. Navigation between screens is managed by SceneManager, a singleton that loads FXML views into the center of a BorderPane shell (SPA-style — no new windows).

Request Flow

User action → @FXML handler in Controller → Service method → Domain model mutation → autoSave() → Controller refreshes ObservableList / chart / label

SPA Navigation (SceneManager)

SceneManager.initStartup() loads startup.fxml into the primary Stage. Once the user selects a farm, loadMainApp() loads main.fxml (containing the sidebar and top-bar shell) and calls navigateTo('dashboard'). Every sidebar button calls navigateTo(route), which looks up the FXML path in the ROUTES map, loads it fresh via FXMLLoader, and sets it as the center of the root BorderPane. A short FadeTransition is played on each page load.

```
+----- App Shell (main.fxml) -----+ | Sidebar
(MainController) | Top-bar (MainController) | | [Dashboard] [Zones] ... | Farm Name | Clock
|                                     Alert                                     Badge                                     |
|-----|-----|-----| | | | Center
(set by SceneManager.navigateTo(route)) | | dashboard.fxml / zones.fxml / animals.fxml / ...
| | | +-----+
```

3. Package: Additional_classes

Contains shared utility classes used across the domain layer. Currently holds the Range class, a simple numeric interval used for sensor thresholds and crop optimal conditions.

Range (Concrete Class)

Package: Additional_classes

PURPOSE

Represents a closed numeric interval [min, max]. Used by NumericSensor subclasses to define normal operating thresholds, and by Crop to store optimal pH and moisture ranges. Enforces that min never exceeds max through constructor and setter validation.

FIELDS

Name	Type	Visibility	Description
min	double	private	Lower bound of the interval
max	double	private	Upper bound of the interval

CONSTRUCTOR

Parameters	Behaviour
double min, double max	Throws IllegalArgumentException if min > max. Sets both fields.

METHODS

Method	Return	Parameters	Logic
getMin	double	—	Returns the lower bound.
setMin	void	double min	Validates min <= max then updates field.
getMax	double	—	Returns the upper bound.
setMax	void	double max	Validates max >= min then updates field.
isInRange	boolean	double value	Returns true if value >= min && value <= max.

RELATIONSHIPS

- Used by → NumericSensor (thresholds), Crop (pH and moisture ranges)

4. Package: Alerts

The Alerts package models the alert subsystem. An abstract Alert base class captures common state (id, type, severity, message, timestamp, resolution). Two concrete subclasses specialise it: HealthAlert (linked to a HealthEvent) and SensorAlert (linked to a SensorReading). AlertManager provides list-management and static filter/sort utilities. Three enums encode the alert taxonomy: AlertType, AlertSeverity, AlertResolution.

Alert (Abstract Class)

Package: Alerts

PURPOSE

Base class for all farm alerts. Generates a UUID on construction, records the current timestamp, and starts in ACTIVE resolution state. Subclasses must implement getZone() to return the zone associated with the alert.

FIELDS

Name	Type	Vis	Description
id	String	private	UUID auto-generated on creation
type	AlertType	private	Category of alert (sensor type or health)
severity	AlertSeverity	private	Warning or Critical
message	String	private	Human-readable description
timestamp	LocalDateTime	private	When the alert was raised
resolution	AlertResolution	private	ACTIVE / ACKNOWLEDGED / RESOLVED / DISMISSED

METHODS

Method	Return	Logic
acknowledge()	void	Sets resolution to ACKNOWLEDGED.
resolve()	void	Sets resolution to RESOLVED.
dismiss()	void	Sets resolution to DISMISSED.
isActive()	boolean	True if resolution == ACTIVE.
isClosed()	boolean	True if RESOLVED or DISMISSED.
getFormattedTimestamp()	String	Formats timestamp as yyyy-MM-dd HH:mm:ss.
getZoneName()	String	Calls getZone(); returns zone name or 'N/A'.
getZone()	ZONE	Abstract — subclass returns the affected zone.

AlertManager (Concrete Class)

Package: Alerts

PURPOSE

Manages two lists of Alert objects: activeAlerts (open alerts) and alertHistory (all-time). Provides lifecycle methods (receive, acknowledge, resolve, dismiss), filter queries, and static utility methods that can be applied to any alert list.

FIELDS

Name	Type	Description
activeAlerts	List<Alert>	Currently open alerts (ACTIVE or ACKNOWLEDGED)
alertHistory	List<Alert>	Full immutable history of every alert ever received

METHODS

Method	Return	Logic
receiveAlert(Alert)	void	Adds to both lists; prints to console.
acknowledgeAlert(id)	void	Finds active alert by id, calls acknowledge().
resolveAlert(id)	void	Finds active alert, calls resolve(), removes from activeAlerts.
dismissAlert(id)	void	Finds active alert, calls dismiss(), removes from activeAlerts.
getCriticalAlerts()	List<Alert>	Filters activeAlerts by AlertSeverity.Critical.
getWarningAlerts()	List<Alert>	Filters activeAlerts by AlertSeverity.Warning.
getAlertsByType(type)	List<Alert>	Filters activeAlerts by AlertType.
getByResolution(res)	List<Alert>	Filters alertHistory by AlertResolution.
getStatsSummary()	String	Returns formatted counts: total, active, critical, warning, resolved, dismissed.
filterBySeverity(list, sev)	List<Alert>	Static. Returns alerts with matching severity.
filterByType(list, type)	List<Alert>	Static. Returns alerts with matching type.
filterByZone(list, zone)	List<Alert>	Static. Returns alerts whose getZone() equals the given zone.
filterByPeriod(list, from, to)	List<Alert>	Static. Returns alerts with timestamp in [from, to].
sortBySeverity(list)	List<Alert>	Static. Critical first, then by timestamp descending.

AlertResolution (Enum)

Package: Alerts

Values: **ACTIVE**, **ACKNOWLEDGED**, **RESOLVED**, **DISMISSED**. Represents the lifecycle state of an alert from creation to closure.

AlertSeverity (Enum)**Package:** Alerts

Values: **Warning**, **Critical**. Warning indicates a condition requiring attention; Critical requires immediate intervention.

AlertType (Enum)**Package:** Alerts

Values: **BioSensorAlert**, **WaterSensorAlert**, **SoilSensorAlert**, **EnvSensorAlert**, **GPS_ESCAPE_ALERT**, **HEALTH_ALERT**. Each value corresponds to the sensor category or event source that generated the alert.

HealthAlert (Concrete Class)**Package:** Alerts**Extends / Implements:** extends Alert**PURPOSE**

An alert triggered by an animal health event (illness, quarantine). Stores a reference to the HealthEvent that triggered it. Validates that the AlertType is exactly HEALTH_ALERT.

Field	Type	Description
triggeringEvent	HealthEvent	The health event that caused this alert
Method	Return	Logic
getZone()	ZONE	Returns triggeringEvent.getAnimal().getZone().
getTriggerReading()	HealthEvent	Returns the stored HealthEvent.

SensorAlert (Concrete Class)**Package:** Alerts**Extends / Implements:** extends Alert**PURPOSE**

An alert triggered by a sensor reading that exceeded its threshold. Stores the SensorReading that caused the breach. Validates that the AlertType is one of the four sensor-type values or GPS_ESCAPE_ALERT.

Field	Type	Description
triggeringEvent	SensorReading	The out-of-range reading that raised the alert
Method	Return	Logic
getZone()	ZONE	Returns triggeringEvent.getSensor().getZone().
getTriggerReading()	SensorReading	Returns the stored SensorReading.

5. Package: Animals

The Animals package models individual farm animals and their health, feeding, and production data. The Animal class is the central entity, carrying inner record classes for production history (WeightRecord, MilkRecord, EggRecord). FeedingProgram encapsulates a scheduled feeding regime. HealthEvent logs a health status change. AnimalHealthStatus is an enum with three states.

Animal (Concrete Class)

Package: Animals

PURPOSE

Represents a single farm animal. Tracks identity (id, name, species), physical attributes (age, weight), health status, optional GPS collar and bio sensors, and production history for milk, eggs, and weight. Three static inner classes (WeightRecord, MilkRecord, EggRecord) represent immutable timestamped production observations.

INNER CLASSES

Class	Fields	Description
WeightRecord	double weight, LocalDateTime timestamp	One weight measurement with timestamp
MilkRecord	double liters, LocalDateTime timestamp	One milk-yield measurement with timestamp
EggRecord	int count, LocalDateTime timestamp	One egg-count measurement with timestamp

FIELDS

Name	Type	Description
id	String	UUID generated at construction
name	String	Animal's display name
species	String	Biological species label (e.g. 'Bovine')
type	LivestockType	RUMINANT or POULTRY
age	int	Age in years
weight	double	Current weight in kg
healthStatus	AnimalHealthStatus	Healthy / Sick / Quarantined
hasGPSCollar	boolean	True when a GPS collar sensor is attached
gpsCollarSensor	GPSCollarSensor	The attached GPS sensor, or null
zone	ZONE	The livestock zone this animal belongs to
healthHistory	List<HealthEvent>	All recorded health events
bioSensors	List<BioSensor>	Attached bio-metric sensors

Name	Type	Description
milkYieldLiters	double	Cumulative milk yield since last reset
eggCount	int	Cumulative egg count since last reset
weightHistory	List<WeightRecord>	All recorded weight measurements
milkHistory	List<MilkRecord>	All recorded milk yield measurements
eggHistory	List<EggRecord>	All recorded egg count measurements

METHODS

Method	Return	Logic
attachGPSCollar(GPSCollarSensor)	void	Sets gpsCollarSensor and hasGPSCollar=true.
attachBioSensor(BioSensor)	void	Appends sensor to bioSensors list.
removeGPSCollar()	void	Nullifies gpsCollarSensor, sets hasGPSCollar=false.
updateWeight(double)	void	Updates weight field and appends a WeightRecord with current time.
updateWeight(double,LocalDateTime)	void	Same but uses supplied timestamp for history record.
addHealthEvent(HealthEvent)	void	Appends to healthHistory and updates healthStatus to event's type.
resolveLastHealthEvent(status)	void	Scans history in reverse for the first unresolved event, marks it resolved, updates healthStatus.
getUnresolvedEvents()	List<HealthEvent>	Returns events where isResolved() is false.
recordMilkYield(double)	void	Accumulates to milkYieldLiters, adds MilkRecord at current time.
recordMilkYield(double,LocalDateTime)	void	Same with explicit timestamp.
recordEgg(int)	void	Accumulates to eggCount, adds EggRecord at current time.
recordEgg(int,LocalDateTime)	void	Same with explicit timestamp.
resetProductionStats()	void	Zeros milkYieldLiters and eggCount; clears milkHistory and eggHistory.
isSick()	boolean	Returns healthStatus == Sick.
isQuarantined()	boolean	Returns healthStatus == Quarantined.

AnimalHealthStatus (Enum)**Package:** Animals

Values: **Healthy**, **Sick**, **Quarantined**. Represents the current overall health condition of an animal. An animal begins Healthy; health events drive transitions.

FeedingProgram (Concrete Class)**Package:** Animals**PURPOSE**

Encapsulates a zone-level feeding schedule: the food type, daily quantity, a list of feeding times (HH:mm strings), and wake/sleep time bounds that prevent feeding outside active hours. Computes whether a feeding is currently due or overdue.

FIELDS

Name	Type	Description
foodType	String	E.g. 'Hay + Grain'
quantity	double	Daily quantity in kg per feeding
schedule	List<String>	Ordered list of HH:mm feeding times
lastFedTime	LocalDateTime	Timestamp of the most recent recordFeeding() call
nextFeedingTime	LocalDateTime	Next computed feeding slot
wakeUpTime	LocalTime	Earliest time feeding is allowed
sleepTime	LocalTime	Latest time feeding is allowed

METHODS

Method	Return	Logic
recordFeeding()	void	Sets lastFedTime to now; recomputes nextFeedingTime.
isDue()	boolean	True if now is past nextFeedingTime and within wake/sleep hours.
isOverdue()	boolean	True if isDue() and more than 30 minutes have passed since nextFeedingTime.
hoursUntilNextFeeding()	long	Duration in hours until nextFeedingTime; 0 if due.
minutesUntilNextFeeding()	long	Same in minutes.
getTimesPerDay()	int	Derived: returns schedule.size().

HealthEvent (Concrete Class)**Package:** Animals**PURPOSE**

Records a single health status change for an animal, including what the status was before and after, a description, and whether it has been resolved. A HealthEvent is considered resolved when statusAfter is non-null at construction or after markResolved() is called.

Field	Type	Description
animal	Animal	The animal this event belongs to
eventType	AnimalHealthStatus	The new health state (Sick, Quarantined, or Healthy for recovery)
statusBefore	AnimalHealthStatus	Health state immediately before this event
statusAfter	AnimalHealthStatus	Health state after resolution (null if unresolved)
date	LocalDateTime	When the event was recorded
description	String	Free-text note about the event
resolved	boolean	True if statusAfter != null at construction or markResolved() called
Method	Return	Logic
markResolved(status)	void	Sets resolved=true and statusAfter to the supplied status.

6. Package: Entities

The Entities package contains the non-animal farm entities: Crop (a cultivated field), AquacultureSpecies (a fish/seafood population group), and supporting enums. Each entity tracks its own production history through inner record classes.

AquacultureSpecies (Concrete Class)

Package: Entities

PURPOSE

Models a single population group of aquatic species within an AquacultureZONE. Tracks population count, harvest records, and mortality. Supports restock cycles: each restock resets the cycleBaseline so that within-cycle survival rates are computed independently of the total operational history.

INNER CLASS — HarvestRecord

Field	Type	Description
weightKg	double	Harvest weight in kilograms
countHarvested	int	Number of individuals harvested
countBefore	int	Population before this harvest
date	LocalDateTime	Timestamp of the harvest

HarvestRecord.getCycleSurvivalRatePercent() computes the percentage of the pre-harvest population that was NOT harvested (i.e. remained alive).

FIELDS

Name	Type	Description
id	String	UUID
name	String	Species name (e.g. 'Tilapia')
initialTotalIndividuals	int	Original stock count — never changes
cycleBaseline	int	Population at the start of the current restock cycle
numSpecies	int	Current live population count
zone	ZONE	Parent aquaculture zone
harvestHistory	List<HarvestRecord >	All harvests ever recorded
cycleStartHarvestIndex	int	Index into harvestHistory marking the start of current cycle

METHODS

Method	Return	Logic
harvest(kg,count)	void	Validates count <= numSpecies, records HarvestRecord, decrements numSpecies.
harvest(kg,count,date)	void	Same with explicit timestamp.
recordMortality(count)	void	Decrement numSpecies; does not add a harvest record.
restock(count)	void	Adds count to numSpecies, resets cycleBaseline, advances cycleStartHarvestIndex.
getCycleMortality()	int	cycleBaseline - numSpecies - harvestedInCycle; never negative.
getTotalMortality()	int	initialTotal - numSpecies - totalHarvested; never negative.
getCycleSurvivalRatePercent()	double	numSpecies / cycleBaseline * 100; capped at 100.
getOverallSurvivalRatePercent()	double	numSpecies / initialTotal * 100; capped at 100.
getTotalHarvestWeightKg()	double	Sum of all HarvestRecord.weightKg values.
getStatusReport()	String	Formatted multi-field summary string.

Crop (Concrete Class)

Package: Entities

PURPOSE

Represents a single cultivated field (crop instance) within a CropZONE. Tracks crop type, variety, planting/expected harvest dates, current growth stage, pH and moisture requirements, and cumulative yield. A YieldRecord inner class stores individual harvest events.

FIELDS

Name	Type	Description
id	String	UUID
cropType	CropType	cereals / vegetables / fruits
variety	String	Cultivar name (e.g. 'Anza')
plantingDate	Date	When seeds were planted
expectedHarvestDate	Date	Target harvest date
growthStage	GrowthStage	Current stage: sowing → germination → growth → maturity → harvest
zone	ZONE	Parent crop zone
optimalPHRange	Range	Acceptable soil pH range

Name	Type	Description
optimalMoistureRange	Range	Acceptable soil moisture % range
yieldKg	double	Cumulative harvested weight
harvestDate	Date	Date of most recent harvest call
boundary	Go GeographicBoundries	Optional polygon boundary for this field
yieldHistory	List<YieldRecord>	All individual harvest records

METHODS

Method	Return	Logic
updateGrowthStage(GrowthStage)	void	Validates non-null, sets growthStage.
isReadyForHarvest()	boolean	True if growthStage == harvest.
recordHarvest(double)	void	Accumulates yieldKg, sets harvestDate to now, adds YieldRecord.
recordHarvest(double, LocalDateTime)	void	Same with explicit timestamp.
wasHarvested()	boolean	True if yieldKg > 0.
getStatusReport()	String	'Crop: variety Stage: stage Type: type'.

CropType (Enum)

Package: Entities

Values: **cereals**, **vegetables**, **fruits**. Classifies the crop category for production reporting and yield-by-type aggregation.

GrowthStage (Enum)

Package: Entities

Values: **sowing**, **germination**, **growth**, **maturity**, **harvest**. Represents the lifecycle stage of a Crop field. Used by simulation and reports.

LivestockType (Enum)

Package: Entities

Values: **RUMINANT**, **POULTRY**. Differentiates the type of livestock zone and animal. RUMINANT animals can produce milk; POULTRY produce eggs.

SoilRequirements (Enum)

Package: Entities

Values: **Optimal_PH**, **Optimal_moisture**, **Both**. Describes which soil conditions are critical for a crop. Used in reporting and sensor alert classification.

7. Package: Farm

Contains a single class, Farm, which is the aggregate root of the entire domain model. All zones, alerts, and report history are owned by and accessed through the Farm instance. It also exposes factory methods for generating all types of zone and farm-level reports.

Farm (Concrete Class)

Package: Farm

PURPOSE

The Farm is the central aggregate root. It holds three typed zone lists (LivestockZONE, CropZONE, AquacultureZONE), an alert list, and four report history lists. Zone-type dispatch uses Java pattern-matching instanceof. The inner FarmStats class is a data-transfer object returned by getStats().

FIELDS

Name	Type	Description
id	String	UUID generated at construction
name	String	Farm display name
location	String	Geographic location string
ownerName	String	Owner/manager name
createdAt	LocalDateTime	Timestamp of farm creation
livestockZones	List<LivestockZONE>	All livestock zones
cropZones	List<CropZONE>	All crop zones
aquacultureZones	List<AquacultureZONE>	All aquaculture zones
alerts	List<Alert>	Complete alert registry for this farm
zoneReports	List<Report>	History of generated zone-level reports
zoneProductionReports	List<ProductionReport>	History of zone production reports
farmReports	List<FarmReport>	History of farm-level summary reports
farmProductionReports	List<FarmProductionReport>	History of farm production reports

KEY METHODS

Method	Return	Logic
addZone(ZONE)	void	Dispatches to the correct typed list using instanceof pattern matching.
removeZone(ZONE)	void	Removes from the matching typed list.

Method	Return	Logic
getZoneByCode(code)	ZONE	Streams getAllZones(), finds first matching code.
getAllZones()	List<ZONE>	Concatenates all three zone lists into an unmodifiable list.
registerAlert(Alert)	void	Adds alert to the alerts list.
getActiveAlerts()	List<Alert>	Returns alerts in ACTIVE or ACKNOWLEDGED state.
getAlertsByZonePeriod(start,end)	List<Alert>	Filters alerts whose timestamp falls in [start,end].
generateReportForZone(zone,type, start,end)	Report	Creates a typed Report subclass based on zone type; stores in history.
generateFarmReport(type,start,end)	FarmReport	Generates reports for all zones, aggregates into FarmReport.
generateFarmProductionReport(type, start,end)	FarmProductionReport	Generates production reports for all zones, aggregates.
getStats()	FarmStats	Computes live counts: zones, animals, sick animals, fields, species, alerts.

INNER CLASS — FarmStats

Public final fields: totalZones, totalAnimals, sickAnimals, totalFields, totalSpecies, totalAlerts, activeAlerts, criticalAlerts. A plain data record with no behaviour.

8. Package: Reports

The Reports package provides a three-tier report hierarchy: Report (abstract zone-level base), FarmReport (aggregated across all zones), ProductionReport (production-specific zone report), and FarmProductionReport (aggregated production across the farm). Three concrete zone report subclasses exist: ReportLiveStockZone, ReportCropZone, ReportAquacultureZone. Three concrete production report subclasses exist: LivestockProductionReport, CropProductionReport, AquacultureProductionReport. ReportType is a supporting enum.

ReportType (Enum)

Package: Reports

Values: **Daily, Weekly, Monthly, Quarterly, Yearly**. Used by ReportService to compute the period start date via LocalDateTime arithmetic.

Report (Abstract Class)

Package: Reports

PURPOSE

Base class for all zone-level reports. The constructor computes alert statistics (total, critical, warning, resolved, unresolved) from the passed list, then delegates to two abstract methods: computeSpecificStats() for domain-specific counts, and computeSensorSummaries() for sensor-threshold summaries.

Field	Type	Description
id	String	UUID
zone	ZONE	The zone this report covers
generatedAt / periodStart / periodEnd	LocalDateTime	Report temporal metadata
reportType	ReportType	Daily / Weekly / ...
totalAlerts / criticalAlerts / warningAlerts	int	Alert counts for the period
resolvedAlerts / unresolvedAlerts	int	Resolution status counts
sensorSummaries	List<String>	Human-readable sensor anomaly descriptions
notes	List<String>	Auto-generated advisory notes

ReportLiveStockZone (Concrete Class)

Package: Reports

Extends / Implements: extends Report

Computes: total, healthy, sick, quarantined animal counts; GPS escape count from alert list; feeding on-time flag from FeedingProgram.isOverdue(). computeSensorSummaries() scans each animal's BioSensors and GPS collar for out-of-threshold readings.

ReportCropZone (Concrete Class)

Package: Reports

Extends / Implements: extends Report

Computes: total fields, ready-for-harvest count, growing count, dominant growth stage (most frequent GrowthStage across all fields). Sensor summaries come from EnvSensor and SoilSensor history.

ReportAquacultureZone (Concrete Class)

Package: Reports

Extends / Implements: extends Report

Computes: species group count, total current individuals, total initial individuals. Sensor summaries from WaterSensor history.

FarmReport (Concrete Class)

Package: Reports

Farm-level aggregated zone report. Iterates over zoneReports, casting each to its concrete type to extract typed stats. Computes GPS escapes from alert types. Generates advisory notes for sick animals, overdue feeding, critical alerts, GPS escapes. Provides a detailed toString() grouped by zone type.

LivestockProductionReport (Concrete Class)

Package: Reports

Extends / Implements: extends ProductionReport

Iterates zone animals, accumulates total milk yield, egg count, and counts producing animals. Computes average milk per producing animal and average eggs per laying animal. Adds a sensor summary entry per producing animal.

CropProductionReport (Concrete Class)

Package: Reports

Extends / Implements: extends ProductionReport

Iterates zone fields, filters those whose harvestDate falls within the report period, accumulates total yield and maps it by CropType. Computes yield per hectare using zone surface. Counts pending (not yet harvested) fields.

AquacultureProductionReport (Concrete Class)

Package: Reports

Extends / Implements: extends ProductionReport

Aggregates harvest weight, current/initial populations, cycle/overall mortality, and average survival rates across all species in the zone. Computes waterQualityScore as (in-range readings / total readings) * 100 from all WaterSensor reading histories.

FarmProductionReport (Concrete Class)

Package: Reports

Top-level production aggregation. Iterates zone production reports (cast by type), sums all livestock, crop, and aquaculture KPIs, computes overall yield/ha and average survival/water quality. Generates farm-wide advisory notes.

9. Package: Sensors

The Sensors package defines the complete sensor hierarchy. Sensor is the abstract base with an auto-generated code, zone reference, status, and reading history. NumericSensor extends it with min/max thresholds and a unit. Four concrete numeric sensor types (BioSensor, EnvSensor, SoilSensor, WaterSensor) and one position sensor (GPSCollarSensor) extend the hierarchy. Two reading classes (NumericSensorReading, GPSSensorReading) extend the abstract SensorReading. Supporting enums encode measure types, status, and severity levels.

Sensor (Abstract Class)

Package: Sensors

PURPOSE

Base for all sensors. Maintains a static integer counter to generate zero-padded 4-digit codes (0001, 0002, ...). Provides reading history management and static utility methods for filtering and sorting reading lists.

Field	Type	Description
code	String	Zero-padded 4-digit auto-generated code
zone	ZONE	The zone this sensor monitors
status	SensorStatus	Active / Faulty / Suspended
readingHistory	List<SensorReading>	All readings recorded on this sensor
Method	Return	Logic
sendReading()	SensorReading	Abstract — subclass creates and stores a new reading.
suspend() / reactivate() / markAsFaulty()	void	Set status field.
addReading(SensorReading)	void	Appends to readingHistory.
filterByPeriod(list,from,to)	List	Static. Returns readings with timestamp in [from,to].
sortByTimestamp(list)	List	Static. Returns sorted copy ascending by timestamp.

NumericSensor (Abstract Class)

Package: Sensors

Extends / Implements: extends Sensor

Adds threshold bounds (minThreshold, maxThreshold) and a measurement unit. isOutOfRange(value) checks whether a value falls outside the interval. lastValue caches the most recent measurement for quick display without scanning history. filterByLevel() is a static utility to filter NumericSensorReading lists by ReadingLevel.

BioSensor (Concrete Class)**Package:** Sensors**Extends / Implements:** extends NumericSensor

Monitors a biological metric (Temperature or ActivityLevel) for a specific animal. The unit is derived automatically from the BioMeasureType (degrees C or steps/min). isAnimalInDistress() delegates to isOutOfRange(lastValue) for quick health checks.

EnvSensor (Concrete Class)**Package:** Sensors**Extends / Implements:** extends NumericSensor

Monitors environmental conditions (Temperature, Humidity, or Rainfall) in a CropZONE. sendReading() creates a NumericSensorReading from lastValue and the configured unit. measure() is a convenience method that returns lastValue.

SoilSensor (Concrete Class)**Package:** Sensors**Extends / Implements:** extends NumericSensor

Monitors soil conditions (PH, Moisture, or Nitrogen) in a CropZONE. Identical structure to EnvSensor, differing only in measure type enum.

WaterSensor (Concrete Class)**Package:** Sensors**Extends / Implements:** extends NumericSensor

Monitors water quality (Temperature or DissolvedOxygen) in an AquacultureZONE. DissolvedOxygen readings below threshold trigger WaterSensorAlert with Critical severity.

GPSCollarSensor (Concrete Class)**Package:** Sensors**Extends / Implements:** extends Sensor**PURPOSE**

Tracks the geographic position of a specific animal. Maintains current latitude and longitude. updateLocation() recomputes isInsideZone by calling the zone's contains() method (ray-casting algorithm via Go geographicBoundaries). sendReading() creates a GPSSensorReading snapshot.

Method	Return	Logic
updateLocation(lat,lon)	void	Updates coordinates, calls checkIfInsideZone() to recompute isInsideZone.
sendReading()	GPSSensorReading	Creates GPSSensorReading with current position and zone-status.
hasEscaped()	boolean	Returns !isInsideZone.

Method	Return	Logic
getLastReading()	GPSSensorReading	Returns the most recent GPSSensorReading from history, or null.

SensorReading (Abstract Class)

Package: Sensors

Base reading class. Stores the sensor reference and an auto-set timestamp. A second constructor accepts an explicit timestamp for historical injection.

NumericSensorReading (Concrete Class)

Package: Sensors

Extends / Implements: extends SensorReading

Stores a numeric value, its unit, and a computed ReadingLevel severity. determineSeverity() compares value to sensor thresholds: if out of range and excess > 20% of range, severity is CRITICAL; otherwise WARNING; otherwise NORMAL.

GPSSensorReading (Concrete Class)

Package: Sensors

Extends / Implements: extends SensorReading

Stores latitude, longitude, and a boolean isInsideZone. isOutsideZone() returns !isInsideZone for convenience.

SensorStatus (Enum)

Package: Sensors

Values: **Active, Faulty, Suspended**. Lifecycle state of a sensor. Faulty sensors are flagged but remain in lists.

ReadingLevel (Enum)

Package: Sensors

Values: **NORMAL, WARNING, CRITICAL**. Severity of a NumericSensorReading determined by how far the value exceeds thresholds.

BioMeasureType (Enum)

Package: Sensors

Values: **Temperature, ActivityLevel**. What a BioSensor measures. Determines unit label automatically.

EnvMeasureType (Enum)

Package: Sensors

Values: **Temperature, Humidity, Rainfall**. What an EnvSensor measures in a crop zone.

SoilMeasureType (Enum)

Package: Sensors

Values: **PH, Moisture, Nitrogen**. What a SoilSensor measures in a crop zone.

WaterMeasureType (Enum)

Package: Sensors

Values: **Temperature, DissolvedOxygen**. What a WaterSensor measures in an aquaculture zone.

AlertSeverity (Sensors pkg) (Enum)

Package: Sensors

Values: **Warning, Critical**. Duplicate of Alerts.AlertSeverity — kept in Sensors package for local use.

10. Package: ZONES

The ZONES package defines the geographic and operational farm zones. ZONE is an abstract base with auto-generated code, name, status, and optional boundaries. Three concrete subclasses specialise it: LivestockZONE (animals, bio sensors, GPS sensors), CropZONE (crop fields, env/soil sensors), and AquacultureZONE (species groups, water sensors). Go geographicBoundries implements polygon point-in-polygon (ray-casting) logic. ZoneStatus is a two-value enum.

ZONE (Abstract Class)

Package: ZONES

PURPOSE

The common base for all farm zones. Uses a static counter to assign sequential 4-digit codes (0001, 0002, ...). Starts in ACTIVE status. Optional Go geographicBoundries can be set post-construction for point-in-polygon testing.

Field	Type	Description
idCounter	int (static)	Shared counter across all ZONE subclasses for unique codes
code	String	Zero-padded 4-digit sequential code
name	String	Zone display name
status	ZoneStatus	ACTIVE or SUSPENDED
boundaries	Go geographicBoundries	Optional polygon boundary; null if not set
Method	Return	Logic
activate() / suspend()	void	Set status to ACTIVE / SUSPENDED.
contains(lat,lon)	boolean	Delegates to boundaries.contains(); false if no boundaries.
hasBoundaries()	boolean	True if boundaries != null.
setBoundaries(Go geographicBoundries)	void	Assigns the polygon boundary.

LivestockZONE (Concrete Class)

Package: ZONES

Extends / Implements: extends ZONE

Holds a list of Animal objects, BioSensor list, GPSCollarSensor list, an optional FeedingProgram, and a LivestockType (RUMINANT or POULTRY). Provides aggregate reading queries (getAllBioReadings, getAllGPSReadings) and production aggregates (getTotalMilkYield, getTotalEggCount). Two constructors: one requires Go geographicBoundries (throws if null), one accepts no boundaries for UI-created zones.

CropZONE (Concrete Class)**Package:** ZONES**Extends / Implements:** extends ZONE

Holds a list of Crop (fields), EnvSensor list, SoilSensor list, and a surfacePlanted area (hectares). Aggregate reading queries: getAllEnvReadings(), getAllSoilReadings(), plus level-filtered versions. getTotalCropYield() sums all field yields.

AquacultureZONE (Concrete Class)**Package:** ZONES**Extends / Implements:** extends ZONE

Holds a list of AquacultureSpecies and WaterSensor list. getTotalHarvestWeight() and getTotalSpeciesCount() aggregate across species. getAllWaterReadings() collects all water sensor histories.

Go GeographicBoundries (Concrete Class)**Package:** ZONES**PURPOSE**

Represents a geographic polygon as an ordered list of [latitude, longitude] pairs. Implements the ray-casting algorithm in contains(lat,lon): a point is inside if a horizontal ray from the point crosses an odd number of polygon edges. Three factory methods create standard shapes: rectangle (4 points), triangle (3 points), and circle (approximated by N points using trigonometry with latitude correction).

Method	Return	Logic
addPoint(lat,lon)	void	Appends [lat,lon] array to the points list.
createRectangle(lat1,lon1,lat2,lon2)	Go GeographicBoundries	Static. Creates a 4-point polygon from two corners.
createTriangle(...)	Go GeographicBoundries	Static. Creates a 3-point polygon.
createCircle(cLat,cLon,radiusM,n)	Go GeographicBoundries	Static. Approximates a circle with n points, correcting longitude radius for latitude.
contains(lat,lon)	boolean	Ray-casting point-in-polygon algorithm.
size()	int	Returns points.size().

ZoneStatus (Enum)**Package:** ZONES

Values: **ACTIVE**, **SUSPENDED**. An ACTIVE zone is operational; a SUSPENDED zone is temporarily taken offline.

11. Layer: Services

All services follow the singleton pattern (private constructor + static getInstance()). Each service delegates to `FarmService.getInstance().getFarm()` for all domain data access, ensuring that a farm switch (selecting a different farm from the startup screen) automatically propagates to all services without re-initialization.

FarmService (Singleton Service)

Package: `com.example.services`

PURPOSE

The primary singleton that holds the active Farm instance. Three factory methods initialise it: `initWithDemo()` loads the built-in `DemoData` farm, `initWithNewFarm()` creates a blank farm, and `initFromSaved()` reconstructs a farm from a `FarmRepository` snapshot. `autoSave()` serialises the current farm state to disk via `FarmRepository` (no-op for the demo farm).

Method	Return	Logic
<code>initWithDemo()</code>	<code>void</code>	Static. Creates <code>DemoData.create()</code> farm, saves a demo entry, sets instance.
<code>initWithNewFarm(name,loc,owner)</code>	<code>void</code>	Static. Creates empty Farm, saves to repository, sets instance.
<code>initFromSaved(SavedFarm)</code>	<code>void</code>	Static. Reconstructs farm from snapshot: zones + animals + optional <code>DataRandomizerService</code> .
<code>getInstance()</code>	<code>FarmService</code>	Throws if not initialised.
<code>autoSave()</code>	<code>void</code>	Serialises zones and animals to <code>FarmRepository</code> (skipped for demo farm).
<code>getTotalSensorCount()</code>	<code>int</code>	Counts all sensors across all zone types.
<code>getStats()</code>	<code>Farm.FarmStats</code>	Delegates to <code>farm.getStats()</code> .

AnimalService (Singleton Service)

Package: `com.example.services`

Provides CRUD operations over animals across all livestock zones. All mutating methods call `fs().autoSave()` after the domain mutation. `resolveHealthEvent()` both resolves the last unresolved event AND adds a new recovery `HealthEvent` to the history (so that the recovery appears as a discrete timeline entry).

Method	Return	Logic
<code>getAllAnimals()</code>	<code>List<Animal></code>	Collects all animals from all livestock zones.
<code>getAnimalsByZone(zone)</code>	<code>List<Animal></code>	Returns animals in the specified zone.

Method	Return	Logic
getAnimalsByHealthStatus(statuses)	List<Animal>	Filters all animals by health status.
getAnimalsByType(type)	List<Animal>	Filters all animals by LivestockType.
addAnimal(name,species,type,age,wt,zone)	Animal	Creates Animal, adds to zone, auto-saves.
removeAnimal(Animal)	void	Finds and removes from its zone, auto-saves.
recordWeight/MilkYield/Egg(...)	void	Delegates to animal, auto-saves.
resolveHealthEvent(animal,statuses)	void	Resolves last event + adds recovery HealthEvent.

ZoneService (Singleton Service)

Package: com.example.services

CRUD and status management for zones. All zone-list queries delegate to Farm. addZone() calls farm.addZone() then autoSave(). setFeedingProgram() and recordFeeding() update livestock zone feeding data.

Method	Return	Logic
getLivestockZones()	List<LivestockZONE>	Direct delegation to farm.
getActiveZoneCount()	int	Counts zones with ZoneStatus.ACTIVE.
addZone(ZONE) / removeZone(ZONE)	void	Delegates to farm, then auto-saves.
renameZone(zone,name)	void	Calls zone.setName(), auto-saves.
setFeedingProgram(zone,fp)	void	Assigns FeedingProgram to zone, auto-saves.
recordFeeding(zone)	void	Calls fp.recordFeeding() if program exists, auto-saves.

SensorService (Singleton Service)

Package: com.example.services

Aggregates and classifies all sensors across the farm. getLastReadingLevel() returns ReadingLevel for any sensor type (GPS escape = CRITICAL). getSensorTypeLabel() produces a human-readable string (e.g. 'Bio · Temperature'). getLastReadingDisplay() formats the last reading value for UI cards.

Method	Return	Logic
getAllSensors()	List<Sensor>	Collects all sensor types from all zones.

Method	Return	Logic
getAllBioSensors() / getAllGPS... / getAllEnv... / ...	typed lists	Collect specific sensor types only.
getActiveSensorCount()	int	Counts sensors with SensorStatus.Active.
getLastReadingLevel(sensor)	ReadingLevel	GPS: checks last reading isInsideZone; Numeric: returns reading severity.
getSensorTypeLabel(sensor)	String	Pattern-matches sensor type, returns 'Type · detail'.
getLastReadingDisplay(sensor)	String	GPS: formatted coordinates; Numeric: value + unit; else 'No data'.

AlertService (Singleton Service)

Package: com.example.services

Thin facade over Farm's alert list. All queries delegate to AlertManager static filters. Provides active alert count for the top-bar badge, and resolution methods for the UI.

Method	Return	Logic
getAllAlerts()	List<Alert>	Returns all alerts sorted by severity (Critical first).
getActiveAlerts()	List<Alert>	Only ACTIVE or ACKNOWLEDGED alerts.
getAlertsByType/Severity/Zone/Period/Resolution(...)	List<Alert>	Various filter delegates.
getActiveAlertCount() / getCriticalAlertCount()	int	Size of filtered lists.
acknowledge/resolve/dismiss(Alert)	void	Direct delegation to alert object.

ReportService (Singleton Service)

Package: com.example.services

Generates reports by computing the period start from the ReportType enum (Daily=-1 day, Weekly=-1 week, Monthly=-1 month, Quarterly=-3 months, Yearly=-1 year) and delegating to Farm's report factory methods.

Method	Return	Logic
generateFarmReport(type)	FarmReport	Computes period, calls farm.generateFarmReport().
generateFarmProductionReport(type)	FarmProductionReport	Same for production.
generateZoneReport(zone,type)	Report	Zone-level report for one zone.

Method	Return	Logic
generateZoneProductionReport(zone,type)	ProductionReport	Zone production report.

DataRandomizerService (Singleton Service)

Package: com.example.services

Populates a newly created empty farm with realistic randomised data: one LivestockZONE with RUMINANT animals (bio sensors, GPS collars, milk records), one CropZONE with cereal and vegetable crops (env and soil sensors), and one AquacultureZONE with two species (water sensors). Used when the user clicks 'Randomize Data' in the startup screen for a new farm.

SimulationService (Singleton Service)

Package: com.example.services

PURPOSE

Implements a discrete-time simulation that advances farm state by one or more days. Each call to simulateDays(n) iterates n days; within each day it simulates every animal, crop, aquaculture species, numeric sensor, and GPS sensor. An inner class SimulationResult accumulates KPIs for the step.

Method	Return	Logic
simulateDays(int days)	SimulationResult	Main entry: loops days, calls all simulateX() helpers, auto-saves.
simulateAnimal(animal,day,result)	void	Weight drift (healthy:+0.2%, sick:-0.3%), milk/egg if applicable, 1% illness/10% recovery chance.
simulateCrop(crop,day,result)	void	Maps elapsed/total days to GrowthStage; harvests on Wednesdays at harvest stage.
simulateAquaculture(sp,day,result)	void	0.05-0.15% daily mortality; harvest every 6 sim-days.
simulateNumericSensor(sensor,day,result)	void	Gaussian reading with 5% spike; creates NumericSensorReading; generates SensorAlert if out of range.
simulateGPS(gps,day,result)	void	Drifts lat/lon by +-0.0002 degrees; calls updateLocation(); generates GPS_ESCAPE_ALERT on inside-to-outside transition.
reset()	void	Resets simulationDate and totalDaysSimulated.
getSimulationDate()	LocalDate	Current simulated date.
getTotalDaysSimulated()	int	Cumulative days simulated since last reset.

SimulationResult fields: List log, int daysSimulated, int healthEvents, double milkLitersTotal, int eggsTotal, int harvestsRecorded, int alertsGenerated, int mortalityCount.

12. Layer: Controllers (JavaFX)

Each controller is a JavaFX FXML controller: the @FXML fields are injected by FXMLLoader, and @FXML-annotated methods are bound to FXML onAction/onKeyReleased attributes. Controllers are stateless between navigation events — they read fresh data from services in initialize() every time the view is loaded.

MainController (FXML Controller)

Package: com.example.controllers

PURPOSE

Controls the persistent app shell (main.fxml): the collapsible sidebar with navigation buttons and the top-bar with farm name, live clock, and alert badge. The sidebar toggles between 64px (icon-only) and 240px (icon + label) using a Timeline animation on the VBox prefWidthProperty.

@FXML Field	Type	UI Role
sidebar	VBox	The collapsible left sidebar container
logoLabel	Label	'FarmManager' brand text (hidden when collapsed)
farmNameLabel	Label	Top-bar farm name from FarmService.getFarmName()
clockLabel	Label	Live clock updated every second
alertBadge	Label	Red badge showing active alert count
btnDashboard/Zones/Animals/...	Button	Navigation buttons, 8 total

Method	Trigger	Logic
initialize()	FXMLLoader	Sets farm name, starts clock, updates badge, marks Dashboard active.
startClock()	internal	Timeline fires every 1s, updates clockLabel with LocalDateTime.now().
updateAlertBadge()	internal	Gets alert count from AlertService; shows/hides badge.
toggleSidebar()	■ button	Animates sidebar width 240px↔64px, toggles logo visibility, updates button text.
setActive(Button)	internal	Removes 'nav-active' CSS class from previous button, adds to new.
navDashboard/Zones/Animals/...()	nav buttons	Calls setActive() then SceneManager.navigateTo(route).

DashboardController (FXML Controller)

Package: com.example.controllers

Displays the main dashboard: four KPI cards (total animals, active zones, active sensors, open alerts), a BarChart of production by zone type (milk L, egg count, harvest kg), a recent-alerts table

(top 5), and quick-action buttons. All data is read from FarmService, ZoneService, SensorService, and AlertService in initialize().

Method	Trigger	Logic
initialize()	FXMLLoader	Calls buildKpis(), buildChart(), buildAlertTable().
buildKpis()	internal	Sets 4 KPI label texts from service counts.
buildChart()	internal	Creates BarChart series for each zone type using aggregated production data.
buildAlertTable()	internal	Populates TableView with top-5 alerts, colour-coded by severity.
navToAnimals/Zones/Reports()	quick-action buttons	Calls SceneManager.navigateTo(route).

ZonesController (FXML Controller)

Package: com.example.controllers

Manages the Zones screen with a TabPane (Livestock | Crop | Aquaculture tabs). Each tab contains a TableView backed by FilteredList. A search field filters by name. Add/Edit open dialogs. The detail panel shows zone boundaries, sensor counts, and zone-type-specific stats. ZoneMapDialog shows a canvas-drawn boundary polygon.

Method	Trigger	Logic
initialize()	FXMLLoader	Builds all three TableViews, wires search filter, loads data.
handleAddZone()	Add button	Opens type-selection dialog, then zone creation dialog.
handleEditZone()	Edit button	Opens edit dialog pre-filled with selected zone data.
handleDeleteZone()	Delete button	Confirms, removes via ZoneService.removeZone(), refreshes list.
showZoneDetail(ZONE)	row selection	Populates detail panel with zone-specific information.
handleViewMap()	Map button	Opens ZoneMapDialog for the selected zone.

AnimalsController (FXML Controller)

Package: com.example.controllers

Displays all animals in a TableView with filter dropdowns (by zone, type, health status). The detail panel shows feeding program, health event timeline, and GPS coordinates if equipped. Dialog buttons open history dialogs for weight, milk, egg, and health event records. Milk/egg/weight history dialogs use ListView with Animal inner record toString() formatting.

Method	Trigger	Logic
initialize()	FXMLLoader	Populates table, sets up filter comboboxes, wires selection listener.

Method	Trigger	Logic
showDetail(Animal)	row click	Fills detail pane: health badge, GPS info, feeding program, history buttons.
handleAddAnimal()	Add button	Opens animal creation dialog, adds via AnimalService.
handleMarkHealth()	Mark Health button	Dialog to log new HealthEvent for selected animal.
handleResolve()	Resolve button	Calls AnimalService.resolveHealthEvent(); refreshes detail.
showWeightHistory()	Weight History button	Dialog with ListView of WeightRecord strings.
showMilkHistory()	Milk History button	Dialog with ListView of MilkRecord strings.
showEggHistory()	Egg History button	Dialog with ListView of EggRecord strings.
showHealthHistory()	Health History button	Dialog with ListView of HealthEvent descriptions.

SensorsController (FXML Controller)

Package: `com.example.controllers`

Displays a scrollable FlowPane of sensor cards. Each card shows the sensor code, type label, last reading, and a colour-coded ReadingLevel badge (green/yellow/red). A filter ComboBox filters by sensor category. Clicking a card opens a SensorChartDialog with a line chart of reading history.

AlertsController (FXML Controller)

Package: `com.example.controllers`

Shows all alerts in a TableView with filter dropdowns for type, severity, and resolution. Selecting a row populates the detail panel with full alert info and resolution status. The 'Mark as Resolved' button calls AlertService.resolve() and refreshes the table.

ReportsController (FXML Controller)

Package: `com.example.controllers`

Left panel: ListView of report types. Right panel: dynamically built content. Supports Farm Report (summary cards + zone table), Farm Production Report (livestock/crop/aquaculture KPIs), and per-zone production reports with trend line charts (milk, eggs, weight, crop yield, aquaculture harvest, GPS inside/outside timeline). Export button writes the current report to a .txt file.

SettingsController (FXML Controller)

Package: `com.example.controllers`

Allows editing farm name, location, and owner. Changes are committed via `FarmService` setters which auto-save. Provides a theme toggle (dark/light) via `SceneManager.applyStylesheet()`. Shows project version and school info in an About section.

SimulationController (FXML Controller)

Package: `com.example.controllers`

Controls the Simulation screen. Three step buttons (+1 Day, +7 Days, +30 Days) call `SimulationService.simulateDays(n)`, then update six KPI labels (milk, eggs, harvests, health events, alerts, mortality) and prepend new log entries to the `ListView` (newest first). Quick-navigation buttons (Reports, Sensors, Animals) jump to those screens.

StartupController (FXML Controller)

Package: `com.example.controllers`

Startup/farm-selection screen. Displays a `ListView` of saved farms loaded from `FarmRepository`. Allows creating a new farm with two distinct initialisation modes, selecting and loading an existing farm, or deleting a saved farm. Calls the appropriate `FarmService` factory method then `SceneManager.loadMainApp()` to transition to the main application shell.

FARM CREATION MODES

Button	Handler	Behaviour
■ Sample Data	<code>createWithSampleData()</code>	Creates the farm then calls <code>DataRandomizerService.populateNewFarm()</code> — produces zones, animals, sensors, and readings ready to explore.
■ Empty Farm	<code>createEmptyFarm()</code>	Creates the farm with no zones or animals — the user builds everything from scratch.
■ Generate Random Farm	<code>generateRandomFarm()</code>	Auto-generates random name/location/owner AND populates the farm with randomised data in one click.

SHARED HELPER

Both `createWithSampleData()` and `createEmptyFarm()` delegate to the private `doCreateFarm(boolean populate)` method. When `populate=true` the method calls `FarmService.markAsRandomized()` before auto-saving, then invokes `DataRandomizerService.populateNewFarm()`. When `populate=false` both calls are skipped and the farm opens completely empty.

BoundaryEditorDialog (Dialog Controller)

Package: `com.example.controllers`

Opens a dialog allowing the user to define a zone's geographic boundary by choosing a shape (Rectangle, Circle, Triangle) and entering coordinate inputs. Calls `GoographicBoundries` factory methods and saves the boundary to the zone via `ZoneService`.

ZoneMapDialog / ZoneAlertHistoryDialog / SensorChartDialog / SensorHistoryDialog (Dialog Controllers)

Package: `com.example.controllers`

ZoneMapDialog: draws the zone's boundary polygon on a JavaFX Canvas. **ZoneAlertHistoryDialog:** lists all alerts filtered to the selected zone. **SensorChartDialog:** shows a LineChart of numeric sensor readings over time; GPS sensors get an inside/outside stepped chart. **SensorHistoryDialog:** displays the raw reading history in a TableView.

13. Layer: Views (FXML)

All FXML files reside in `src/main/resources/com/example/views/`. They declare the UI layout; controller classes are referenced via `fx:controller` attributes. No business logic appears in FXML — all onAction handlers are @FXML methods in the controller.

startup.fxml (FXML View)

Package: `resources/com/example/views/`

Extends / Implements: Controller: `StartupController`

Two-panel VBox layout. Left panel: ListView of saved farms with Open/Delete actions per row. Right panel: GridPane form (Name, Location, Owner fields), then a labelled pair of creation buttons — '■ Sample Data' and '■ Empty Farm' — plus a '■ Generate Random Farm' button below. Selecting Sample Data pre-populates the farm via `DataRandomizerService`; Empty Farm skips population and opens a blank farm.

KEY FX:ID COMPONENTS

fx:id	Type	Role
farmList	ListView	Saved farm entries with Open/Delete buttons
newFarmName newFarmLocation newFarmOwner	TextField	New farm input fields
errorLabel	Label	Validation error message (hidden until triggered)

main.fxml (FXML View)

Package: `resources/com/example/views/`

Extends / Implements: Controller: `MainController`

BorderPane shell. Left contains sidebar VBox with logo, toggle button, and 8 navigation buttons. Top contains HBox topbar with `farmNameLabel`, spacer, `clockLabel`, bell button, `alertBadge`. Center is left empty and set programmatically by `SceneManager.navigateTo()`.

KEY FX:ID COMPONENTS

fx:id	Type	Role
sidebar	VBox	Collapsible navigation sidebar
logoLabel	Label	Brand label hidden when collapsed
farmNameLabel	Label	Farm name in topbar
clockLabel	Label	Live clock
alertBadge	Label	Alert count badge
btnDashboard/Zones/Animals/...	Button x8	Navigation buttons

dashboard.fxml (FXML View)**Package:** resources/com/example/views/**Extends / Implements:** Controller: DashboardController

ScrollPane > VBox. Header row with page title. Four KPI cards in HBox. BarChart for production. TableView for recent alerts. Quick-action HBox.

KEY FX:ID COMPONENTS

fx:id	Type	Role
kpiAnimals/kpiZones/kpiSensors/kpiAlerts	Label	KPI numeric values
productionChart	BarChart	Production by zone type
alertsTable	TableView	Top-5 recent alerts

zones.fxml (FXML View)**Package:** resources/com/example/views/**Extends / Implements:** Controller: ZonesController

ScrollPane > VBox. TabPane with three tabs (Livestock, Crop, Aquaculture). Each tab: search HBox, TableView, toolbar buttons. Right detail panel in SplitPane.

KEY FX:ID COMPONENTS

fx:id	Type	Role
tabPane	TabPane	Three zone-type tabs
livestockTable/cropTable/aquaTable	TableView	Zone tables
searchField	TextField	Live name filter
detailPanel	VBox	Zone detail on right side

animals.fxml (FXML View)**Package:** resources/com/example/views/**Extends / Implements:** Controller: AnimalsController

ScrollPane > VBox. Filter bar (zone, type, health ComboBoxes). TableView of animals. Detail VBox on right with health badge, GPS info, history buttons.

KEY FX:ID COMPONENTS

fx:id	Type	Role
animalTable	TableView	Full animal list
zoneFilter/typeFilter/healthFilter	ComboBox	Filter dropdowns
detailPane	VBox	Selected animal details

sensors.fxml (FXML View)**Package:** resources/com/example/views/**Extends / Implements:** Controller: SensorsController

ScrollPane > VBox. Filter ComboBox. FlowPane of dynamically built sensor cards. Each card is a VBox with code, type label, last reading, and severity badge.

KEY FX:ID COMPONENTS

fx:id	Type	Role
sensorFlow	FlowPane	Dynamic grid of sensor cards
typeFilter	ComboBox	Filter by sensor category

alerts.fxml (FXML View)**Package:** resources/com/example/views/**Extends / Implements:** Controller: AlertsController

ScrollPane > VBox. Filter HBox (type, severity, resolution ComboBoxes). TableView of all alerts. Detail VBox below showing selected alert's full information.

KEY FX:ID COMPONENTS

fx:id	Type	Role
alertsTable	TableView	Full alert list
typeFilter/severityFilter/resolutionFilter	ComboBox	Filters
detailPanel	VBox	Selected alert details

reports.fxml (FXML View)**Package:** resources/com/example/views/**Extends / Implements:** Controller: ReportsController

SplitPane. Left: ListView of report types. Right: ScrollPane > VBox dynamically populated with report content (KPI cards, tables, line charts). Export button at the bottom.

KEY FX:ID COMPONENTS

fx:id	Type	Role
reportTypeList	ListView	Report type options
reportContent	VBox	Dynamically built report output
exportBtn	Button	Export to .txt file

settings.fxml (FXML View)**Package:** resources/com/example/views/**Extends / Implements:** Controller: SettingsController

ScrollPane > VBox. Farm info edit section (TextFields for name/location/owner, Save button). Theme toggle (RadioButtons dark/light). About section (project info labels).

KEY FX:ID COMPONENTS

fx:id	Type	Role
farmNameField/locationField/ownerField	TextField	Editable farm info
darkTheme/lightTheme	RadioButton	Theme selection
versionLabel	Label	App version display

simulation.fxml (FXML View)

Package: resources/com/example/views/

Extends / Implements: Controller: SimulationController

ScrollPane > VBox. Header with sim date and day count. Step controls (+1/+7/+30 Day buttons, Reset). KPI summary HBox (6 cards). Quick-nav buttons (Reports, Sensors, Animals). Activity log ListView.

KEY FX:ID COMPONENTS

fx:id	Type	Role
simDateLabel/simDayLabel	Label	Current simulation date and day count
kpiMilk/kpiEggs/kpiHarvests/kpiHealthEvents/kpiAlerts/kpiMortality	Label	Step KPIs
activityLog	ListView	Chronological event log (newest first)

14. Layer: Styles (CSS)

The application ships with two CSS files: `main.css` (dark theme, loaded by default) and `light.css` (light overrides toggled via `SettingsController`). A `dark.css` file duplicates `main.css` for explicit switching.

Dark Theme Colour Palette

Token	Hex	Used For
slate-900	#0f172a	Root background, page background
slate-800	#1e293b	Sidebar, card surfaces
green-500	#22c55e	Primary accent: buttons, active states, borders
sky-500	#0ea5e9	Secondary accent: info badges, links
text-primary	#f1f5f9	Main text colour
text-muted	#94a3b8	Secondary labels, placeholders
danger	#ef4444	Critical alerts, error states
warning	#f59e0b	Warning alerts, caution indicators
card-bg	#1e293b	Content card background
topbar-bg	#0f172a	Top navigation bar background
sidebar-bg	#1e293b	Left sidebar background

Key CSS Class Definitions

Class	Applied To	Effect
.root-pane	BorderPane root	Sets background-color to slate-900.
.sidebar	VBox sidebar	Background slate-800, no border radius.
.sidebar-logo	Label	Large green brand label, padding 20px.
.nav-btn	Button	Transparent background, muted text; full-width; left-aligned.
.nav-active	Button (active)	Green-500 background, white bold text.
.topbar	HBox	Slate-900 background, bottom border, fixed height ~52px.
.topbar-title	Label	White bold 16pt font for farm name.
.topbar-clock	Label	Muted text 11pt monospace-style.
.topbar-alert-badge	Label	Red pill badge: danger background, white text, 10px border-radius.
.content-root	VBox	Page content wrapper, padding 24px all sides.

Class	Applied To	Effect
.page-title	Label	Bold 22pt white text for page headings.
.section-title	Label	Green-500 bold 11pt uppercase labels.
.kpi-card	VBox	Slate-800 rounded card, 12px radius, padding 16px.
.kpi-value	Label	Bold 24pt white number.
.kpi-label	Label	Muted 10pt label below value.
.btn-primary	Button	Green-500 filled, white text, 8px radius, hover darkens.
.btn-secondary	Button	Slate-700 filled, muted text, 8px radius.
.badge-healthy	Label	Green pill badge.
.badge-sick	Label	Red pill badge.
.badge-quarantined	Label	Orange pill badge.
.badge-normal	Label	Green pill badge for sensor reading level.
.badge-warning	Label	Yellow pill badge.
.badge-critical	Label	Red pill badge.

Light Theme (light.css)

Overrides background colours to white/light-grey tones, text colours to near-black, and card backgrounds to #f8fafc. Green accent is retained. Applied by SettingsController via SceneManager.applyStylesheet('/com/example/styles/light.css').

15. Utilities and Entry Point

App (JavaFX Application Entry Point)

Package: `com.example`

Extends `javafx.application.Application`. The `start()` method calls `SceneManager.initStartup()` to load the startup screen, sets the window title, enforces minimum dimensions (920x600), and shows the stage. The static `main()` method calls `launch(args)` to initialise the JavaFX platform and invoke `start()` on the FX Application Thread.

SceneManager (Singleton Utility)

Package: `com.example.utils`

PURPOSE

Central navigation hub for the SPA-style single-window application. Manages the JavaFX Stage and root `BorderPane`. Holds a static `ROUTES` map mapping string route names to FXML resource paths. A comment block at the top of the file documents the four-step workflow for adding new screens.

Method	Return	Logic
<code>initStartup(Stage)</code>	<code>void</code>	Static. Creates instance, loads <code>startup.fxml</code> into a new Scene (920x600), applies CSS.
<code>loadMainApp()</code>	<code>void</code>	Loads <code>main.fxml</code> into a new Scene (1280x780), preserves fullscreen state, calls <code>navigateTo('dashboard')</code> .
<code>navigateTo(route)</code>	<code>void</code>	Looks up FXML path in <code>ROUTES</code> , loads via <code>FXMLLoader</code> , plays 180ms <code>FadeTransition</code> , sets as root center.
<code>navigateToStartup()</code>	<code>void</code>	Transitions back to the startup screen; resets min dimensions.
<code>applyStylesheet(path)</code>	<code>void</code>	Clears and replaces scene stylesheets (used for theme switching).
<code>getInstance()</code>	<code>SceneManager</code>	Returns the singleton.
<code>getRoot()</code>	<code>BorderPane</code>	Returns the main shell <code>BorderPane</code> .

Route Registry

Route Key	FXML File	Screen Purpose
<code>dashboard</code>	<code>dashboard.fxml</code>	Main KPI overview
<code>zones</code>	<code>zones.fxml</code>	Zone management (tabbed)
<code>animals</code>	<code>animals.fxml</code>	Animal management

Route Key	FXML File	Screen Purpose
sensors	sensors.fxml	Sensor grid with readings
alerts	alerts.fxml	Alert centre
reports	reports.fxml	Report generation
simulation	simulation.fxml	Time-step simulation
settings	settings.fxml	Settings and about

FarmRepository (Utility Class)

Package: `com.example.utils`

Persists farm metadata to `~/smartfarm/farms.properties` using Java's Properties API (no external library required). The flat key schema prefixes all entries with 'farm.i.' where i is the farm index. Zones use 'farm.i.zone.j.*' and animals use 'farm.i.animal.j.*'. `save()` performs an upsert (replace by id or append). `delete()` removes by id.

Inner classes: `SavedFarm` (id, name, location, owner, createdAt, isDemo, wasRandomized, List, List), `SavedZone` (type, name, IstType), `SavedAnimal` (name, species, type, age, weight, zoneName, health).

RandomFarmGenerator (Utility Class)

Package: `com.example.utils`

Provides random name lists (farm names, animal names, zone names) and helper methods used by `DataRandomizerService` to build realistic farm data for new user farms.

DemoData (Utility / Seed Data Class)

Package: `Menu`

A static factory class with a single `create()` method that returns a fully populated demo Farm. The demo includes: North Pasture (LivestockZONE, 3 animals with bio sensors, GPS collars, health events, feeding program), South Fields (CropZONE, 3 crops at different growth stages, env and soil sensors), Pond Alpha (AquacultureZONE, tilapia and salmon, water sensors). Historical readings are injected via private `injectNumericReadings()` and `injectGPSReadings()` helpers that create 15 timestamped readings spaced one day apart.

16. Class Relationship Summary

Class	Depends On	Depended on By
Farm	LivestockZONE, CropZONE, AquacultureZONE, Alert, Report subtypes	FarmService, FarmReport, ReportService
ZONE (abstract)	GoegraphicBoundries	Farm, LivestockZONE, CropZONE, AquacultureZONE
LivestockZONE	Animal, BioSensor, GPSCollarSensor, FeedingProgram	Farm, AnimalService, ZoneService, ReportLiveStockZone
CropZONE	Crop, EnvSensor, SoilSensor	Farm, ZoneService, ReportCropZone
AquacultureZONE	AquacultureSpecies, WaterSensor	Farm, ZoneService, ReportAquacultureZone
Animal	LivestockType, AnimalHealthStatus, HealthEvent, BioSensor, GPSCollarSensor, WeightRecord, MilkRecord, EggRecord	LivestockZONE, AnimalService, HealthAlert
Sensor (abstract)	ZONE, SensorReading	LivestockZONE, CropZONE, AquacultureZONE, SensorService
NumericSensor	—	BioSensor, EnvSensor, SoilSensor, WaterSensor, NumericSensorReading
GPSCollarSensor	Animal, GPSSensorReading	LivestockZONE, Animal, SensorService
Alert (abstract)	AlertType, AlertSeverity, AlertResolution, ZONE	Farm, AlertService, AlertsController
SensorAlert	SensorReading	Farm, SimulationService
HealthAlert	HealthEvent	Farm, AnimalService
Report (abstract)	ZONE, Alert	Farm, FarmReport, ReportService
FarmService	Farm, FarmRepository, DemoData, DataRandomizerService	All services and controllers (via getInstance())
SceneManager	Stage, BorderPane, FXMLLoader	App, MainController, all nav methods
FarmRepository	Properties, Path	FarmService, StartupController
SimulationService	FarmService, all domain classes	SimulationController

17. Data Flow Examples

Scenario A — Farm Manager Adds a New Crop Zone

1	User fills zone name in the Add Zone dialog in zones.fxml.
2	ZonesController.handleAddZone() reads the dialog result.
3	Controller calls ZoneService.addZone(new CropZONE(name)).
4	ZoneService delegates to farm.addZone(zone), which adds to cropZones list.
5	ZoneService calls FarmService.autoSave(), writing the new zone to farms.properties.
6	Controller clears and reloads the ObservableList; new zone appears in the table.
7	User can now click the zone to see the detail panel or add crops/sensors.

Scenario B — A Soil Sensor Reading Exceeds its Threshold

1	SimulationService.simulateNumericSensor(soilSensor, day, result) is called during a simulation step.
2	A random reading value is generated; it exceeds the sensor's maxThreshold.
3	A NumericSensorReading is created; determineSeverity() classifies it as WARNING or CRITICAL.
4	An instanceof check identifies it as a SoilSensor; a SensorAlert with AlertType.SoilSensorAlert is created.
5	farm.registerAlert(alert) adds the alert to the farm's alert list.
6	result.alertsGenerated is incremented; the event is appended to result.log.
7	After the simulation step, SimulationController updates kpiAlerts label.
8	Navigating to Alerts page shows the new alert in the table with CRITICAL badge.

Scenario C — Farm Manager Generates a Production Report

1	User selects 'Farm Production — Monthly' in the reports.fxml ListView.
2	ReportsController calls ReportService.generateFarmProductionReport(ReportType.Monthly).
3	ReportService computes periodStart = now.minusMonths(1), calls farm.generateFarmProductionReport().
4	Farm calls generateAllZoneProductionReports(), creating LivestockProductionReport, CropProductionReport, AquacultureProductionReport for each zone.
5	Each zone report iterates its entities (animals / crops / species) accumulating production KPIs.

6	FarmProductionReport aggregates all zone reports, computes averages, generates advisory notes.
7	ReportsController receives the FarmProductionReport and renders KPI cards + zone breakdown table in reportContent VBox.
8	User clicks Export; controller calls FileChooser, writes report.toString() to the selected .txt file.